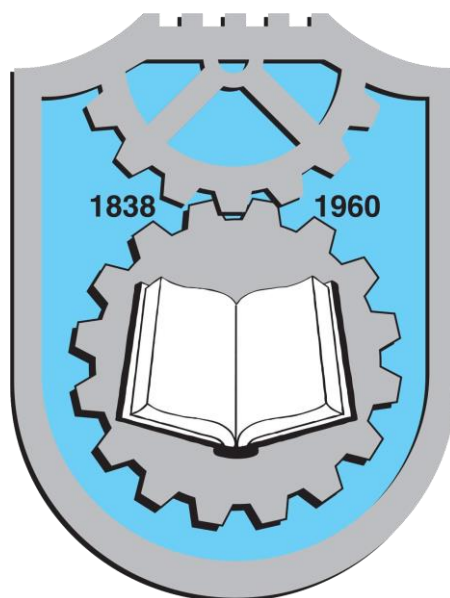


Univerzitet u Kragujevcu
Fakultet inženjerskih nauka



Predmet: Vestačka inteligencija

Tema: Klasifikacija automobila primenom neuronske mreže i genetskog algoritma

Student:

Anita Mijatovic 612/2022

Predmetni profesor:

Vesna Rankovic

SADRŽAJ

APSTRAKT	4
1. UVOD	4
1.1 Cilj rada	4
2. OPIS PROBLEMA I SKUPA PODATAKA	5
2.1 Car Evaluation Dataset	5
2.2 Ulazni atributi	5
2.3 Izlazni atribut – Klase	5
2.4 Vizualizacija podataka	6
2.5 Priprema podataka	7
3. KORIŠĆENI ALGORITMI	8
3.1 Višeslojni perceptron (MLP)	8
3.2 Funkcije aktivacije	9
3.3 Algoritam učenja – Backpropagation i Adam	9
3.4 Regularizacija – L2	9
3.5 Genetski algoritam	9
3.6 Prostor pretrage hiperparametara	10
3.7 Operatori genetskog algoritma	10
3.8 Opravdanje izbora metoda	10
4. IMPLEMENTACIJA	11
4.1 Korišćene biblioteke	11
4.2 Struktura programa	11
4.3 Izvorni kod	12
5. REZULTATI I DISKUSIJA	20
5.1 Rezultati genetskog algoritma	20
5.2 Performanse finalnog modela	21
5.3 Analiza po klasama	22
5.4 Poređenje sa alternativnim metodama	24
5.5 Analiza grešaka modela	26
6. PREDNOSTI I MANE KORIŠĆENE METODE	27
6.1 Prednosti	27
6.2 Mane	28
7. VERIFIKACIJA U MATLAB-u	29
7.1 Opis MATLAB implementacije	29
7.2 Arhitektura MATLAB mreže	29

7.3 Izvorni kod (MATLAB)	29
7.4 Rezultati verifikacije	33
7.5 Diskusija verifikacije	34
8. ZAKLJUČAK	35
LITERATURA	36

APSTRAKT

Ovaj seminarski rad bavi se rešavanjem problema klasifikacije automobila primenom metoda veštačke inteligencije. Kao skup podataka korišćen je Car Evaluation Dataset preuzet sa UCI Machine Learning Repository-ja, koji sadrži 1728 instanci opisanih sa šest kategorijskih atributa i jednim izlaznim atributom koji predstavlja ocenu automobila. Problem je rešen kombinacijom dve metode veštačke inteligencije: višeslojnog perceptrona (MLP – Multilayer Perceptron) kao klasifikatora i genetskog algoritma (GA) za automatsku optimizaciju hiperparametara neuronske mreže. Eksperimentalni rezultati pokazuju da GA-optimizovani MLP model postiže tačnost klasifikacije iznad 95%, čime se potvrđuje opravdanost primene evolutivnih algoritama u optimizaciji parametara modela mašinskog učenja.

1. UVOD

Veštačka inteligencija (VI) je oblast računarskih nauka koja se bavi kreiranjem sistema sposobnih da obavljaju zadatke koji normalno zahtevaju ljudsku inteligenciju – kao što su prepoznavanje obrazaca, donošenje odluka i učenje iz iskustva. U poslednjih nekoliko decenija, metode mašinskog učenja, posebno neuronske mreže, dokazale su svoju efikasnost u rešavanju kompleksnih problema klasifikacije i regresije.

Neuronske mreže su inspirisane strukturom biološkog mozga i sastoje se od međusobno povezanih čvorova (neurona) organizovanih u slojeve. Njihova sposobnost aproksimacije nelinearnih funkcija čini ih posebno pogodnim za probleme sa složenim granicama odlučivanja.

Međutim, performanse neuronske mreže u velikoj meri zavise od izbora hiperparametara: broja i veličine skrivenih slojeva, funkcije aktivacije, brzine učenja i regularizacionih koeficijenata. Ručno podešavanje ovih parametara je vremenski zahtevno i suboptimalno. Genetski algoritmi, inspirisani principima biološke evolucije, predstavljaju efikasan pristup automatskog pretraživanja prostora hiperparametara.

U okviru ovog rada, primenjena je kombinacija ove dve metode na problemu evaluacije automobila. Dataset Car Evaluation sa UCI repozitorijuma odabran je kao benchmark problem jer sadrži jasno definisane kategorijske attribute i poznatu distribuciju klasa, što ga čini pogodnim za verifikaciju algoritama klasifikacije.

1.1 Cilj rada

Cilj ovog seminarskog rada je implementacija i evaluacija hibridnog sistema koji koristi genetski algoritam za optimizaciju hiperparametara MLP neuronske mreže, primenjenog na problem klasifikacije automobila. Specifični ciljevi su:

- Analiza i vizualizacija skupa podataka Car Evaluation

- Implementacija MLP klasifikatora u programskom jeziku Python
- Implementacija genetskog algoritma za pretragu optimalnih hiperparametara
- Poređenje performansi optimizovanog modela sa baseline modelom
- Diskusija rezultata i izvođenje zaključaka

2. OPIS PROBLEMA I SKUPA PODATAKA

2.1 Car Evaluation Dataset

Car Evaluation Dataset je jedan od klasičnih skupova podataka u oblasti mašinskog učenja, dostupan na adresi <https://archive.ics.uci.edu/ml/datasets/Car+Evaluation>. Dataset je originalno kreiran 1997. godine od strane M. Bohaneca i u literaturi se često koristi kao benchmark za testiranje algoritama klasifikacije.

Skup podataka sadrži ukupno **1728 instanci**, bez nedostajućih vrednosti. Svaka instanca opisuje jedan automobil sa šest ulaznih atributa i jednim izlaznim atributom (klasom). Dataset predstavlja potpun faktorijski dizajn – pokrivene su sve moguće kombinacije vrednosti ulaznih atributa.

2.2 Ulazni atributi

Svih šest ulaznih atributa je kategorijskog (ordinalnog) tipa:

Atribut	Moguće vrednosti	Opis
buying	vhigh, high, med, low	Cena kupovine automobila
maint	vhigh, high, med, low	Cena održavanja automobila
doors	2, 3, 4, 5more	Broj vrata
persons	2, 4, more	Kapacitet putnika
lug_boot	small, med, big	Veličina gepeka
safety	low, med, high	Procenjena bezbednost automobila

Slika 1-prikaz atributa kategorijskog tipa

2.3 Izlazni atribut – Klase

Izlazni atribut (class) predstavlja ukupnu ocenu automobila i može uzeti jednu od četiri vrednosti:

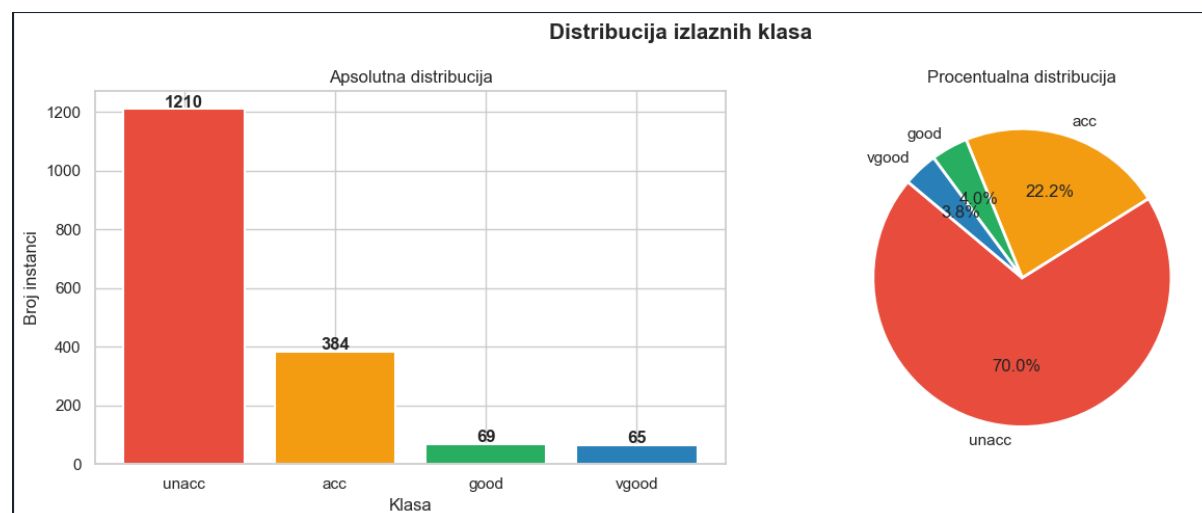
Klasa	Udeo u datasetu	Značenje
unacc	~70.0% (1210 instanci)	Neprihvatljivo
acc	~22.2% (384 instance)	Prihvatljivo
good	~3.99% (69 instanci)	Dobro
vgood	~3.76% (65 instanci)	Veoma dobro

Slika 2-izlazni atributi i razlicite vrednosti za njih

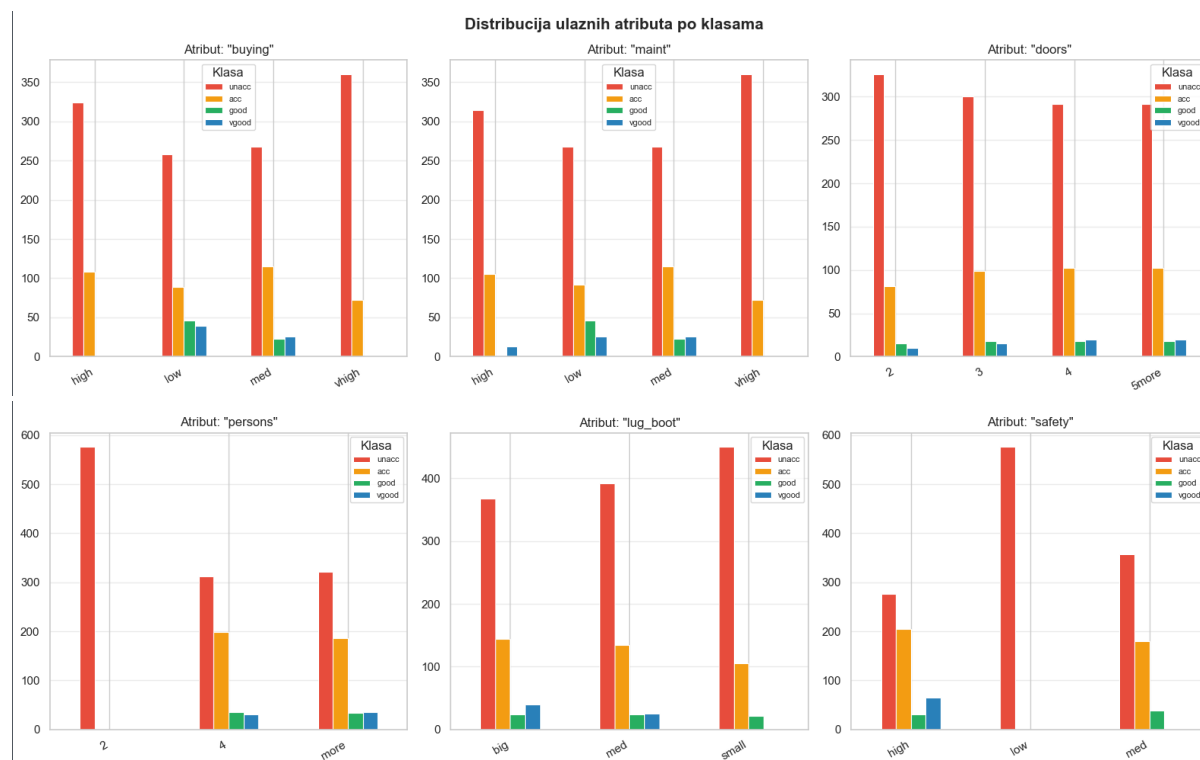
Izražena neravnoteža klasa (dominantna klasa unacc sa ~70%) predstavlja dodatni izazov za algoritme klasifikacije.

2.4 Vizualizacija podataka

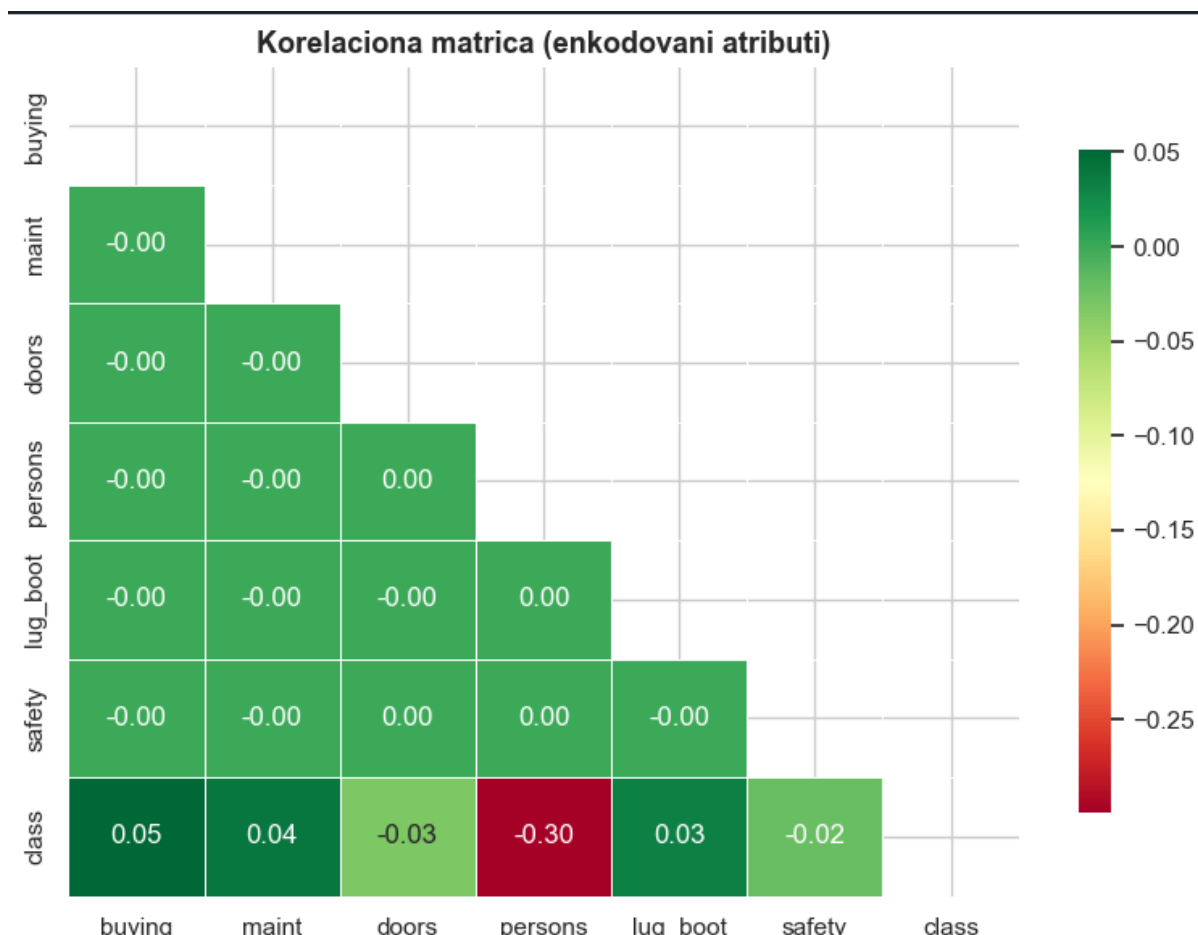
U okviru implementacije izvršena je detaljna vizualizacija skupa podataka pokrenuta Python skriptom. Generisane su sledeće vizualizacije:



Slika 3– Distribucija klasa (bar chart i pie chart) – prikazuje dominaciju klase unacc



Slika 4 – Distribucija svakog ulaznog atributa po klasama – otkriva da atributi safety i persons imaju najveći uticaj na klasu



Slika 5 – Korelaciona toplotna mapa – potvrđuje da su atributi međusobno nisko korelisani, što znači da svaki donosi nezavisnu informaciju

2.5 Priprema podataka

Budući da su svi atributi kategorijskog tipa sa prirodnim ordinalnim redosledom, primenjena je **ordinalna enkodovana shema**. Svaka vrednost atributa preslikana je u ceo broj koji odražava njen relativni položaj na skali:

Attribute	Condition	Encoded Value
buying / maint	low, med, high med, vhigh	0, 1, 2, 3
doors	2, 3, 4, 5more	0, 1, 2, 3
persons	2, 4, more	0, 1, 2
lug_boot	small, med, big	0, 1, 2
safety	low, med, high	0, 1, 2

Nakon enkodovanja primenjena je **standardizacija** (StandardScaler) koja transformiše svaki atribut na nultu srednju vrednost i jediničnu standardnu devijaciju. Podaci su podeljeni na skup

za treniranje (80%, 1382 instance) i skup za testiranje (20%, 346 instanci), uz stratifikaciju po klasi.

3. KORIŠĆENI ALGORITMI

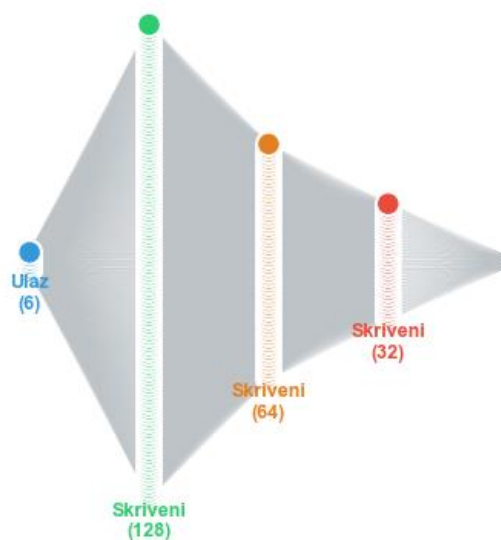
3.1 Višeslojni perceptron (MLP)

Višeslojni perceptron (eng. Multilayer Perceptron, MLP) je osnovna arhitektura feedforward neuronske mreže. Sastoji se od ulaznog sloja, jednog ili više skrivenih slojeva i izlaznog sloja. Svaki neuron prima ulaze iz svih neurona prethodnog sloja, ponderisane težinama, sabira ih sa bias-om i primenjuje nelinearnu funkciju aktivacije.

Arhitektura za ovaj problem:

- Ulazni sloj: 6 neurona (po jedan za svaki atribut)
- Skriveni sloj(evi): optimizovano genetskim algoritmom
- Izlazni sloj: 4 neurona sa softmax aktivacijom (po jedan za svaku klasu)

Arhitektura neuronske mreže



Slika 6 – Arhitektura GA-optimizovane neuronske mreže ($6 \rightarrow 128 \rightarrow 64 \rightarrow 32 \rightarrow 4$)

Matematički, izlaz neurona u sloju l računa se kao:

$$a^{(l)} = f(W^{(l)} \cdot a^{(l-1)} + b^{(l)})$$

Equation for neural network layer activation and ' W ', ' b ', and ' a ' are weights, bias, and activations respectively.

gde je $W(l)$ matrica težina, $b(l)$ vektor pragova, a $f(\cdot)$ funkcija aktivacije.

3.2 Funkcije aktivacije

U eksperimentima su razmatrane tri funkcije aktivacije:

ReLU (Rectified Linear Unit): $f(x) = \max(0, x)$ Najčešće korišćena funkcija aktivacije u dubokim mrežama. Nema problema sa nestajanjem gradijenta za pozitivne vrednosti, brza konvergencija.

Tanh (hiperbolni tangens): $f(x) = (e^x - e^{-x}) / (e^x + e^{-x})$ Izlaz centriran oko nule, u opsegu $(-1, 1)$. Pogodna za dublje mreže jer pomaže normalizaciji aktivacija.

Logistička (sigmoid): $f(x) = 1 / (1 + e^{-x})$ Izlaz u opsegu $(0, 1)$. Može patiti od nestajanja gradijenta za velike apsolutne vrednosti ulaza.

3.3 Algoritam učenja – Backpropagation i Adam

MLP se trenira algoritmom **backpropagation** u kombinaciji sa **Adam optimizatorom**. Funkcija gubitka je cross-entropy za višeklasnu klasifikaciju:

$$L = -(1/N) \cdot \sum \sum y_{ij} \cdot \log(\hat{y}_{ij})$$

Equation for Cross-Entropy Loss, where 'y' is the true label and 'ŷ' is the predicted probability.

gde je N broj instanci, y_{ij} stvarna, a $\hat{y}_{ij}$ predviđena verovatnoća klase j za instancu i.

Backpropagation algoritam izračunava gradijent funkcije gubitka po svim težinama mreže primenom lančanog pravila diferencijacije, a Adam optimizator adaptivno podešava korake učenja za svaki parametar zasebno.

3.4 Regularizacija – L2

Kako bi se sprečilo preprilagođavanje (overfitting), korišćena je **L2 regularizacija** (weight decay). Regularizacioni koeficijent alpha dodat je funkciji gubitka:

$$L_{\text{reg}} = L + \alpha \cdot \sum w^2$$

where L is the base loss, α is the regularization strength, $\sum w^2$ is the L2 norm of model weights, and L_{reg} is the total regularized loss.

Vrednost alpha je takođe hiperparametar koji se optimizuje genetskim algoritmom.

3.5 Genetski algoritam

Genetski algoritam (GA) je metaheuristički algoritam pretrage inspirisan principima Darwinove evolucije – prirodnom selekcijom, ukrštanjem i mutacijom. Koristi se za pretragu

prostora hiperparametara MLP-a jer je taj prostor diskretan, visokodimenzijsan i nije pogodna za gradijentne metode pretrage.

Analogija sa biologijom:

Pojam u GA	Značenje u kontekstu problema			
Jedinka (hromozom)	Jedan specifičan skup hiperparametara MLP mreže			
Populacija	Skup svih kandidatnih rešenja (skupova hiperparametara)			
Fitness funkcija	Tačnost klasifikacije modela procenjena unakrsnom validacijom (cross-validation)			
Selekcija	Preferiranje i odabir jedinki sa višim fitness-om za sledeću generaciju			
Ukrštanje (Crossover)	Kombinovanje hiperparametara izabrana dva "roditelja" radi kreiranja potomka			
Mutacija	Nasumična promena vrednosti jednog hiperparametra radi održavanja raznolikosti			

3.6 Prostor pretrage hiperparametara

Genetski algoritam pretražuje sledeći prostor hiperparametara:

Hiperparametar	Moguće vrednosti	Opis / Uloga parametra		
hidden_layer_sizes	(32,), (64,), (128,), (64,32), (128,64), (64,64), (128,64,32), (64,32,16), (100,50)	Broj skrivenih slojeva i broj neurona u svakom sloju		
activation	relu, tanh, logistic	Aktivaciona funkcija za skrivene slojeve		
alpha (L2)	0.0001, 0.0001, 0.01, 0.1	Koeficijent L2 regularizacije (kazna za velike težine)		
learning_rate_init	0.001, 0.005, 0.01	Početna stopa učenja za optimizaciju		
max_iter	300, 500, 700	Maksimalan broj epoha (iteracija) za treniranje mreže		

3.7 Operatori genetskog algoritma

Inicijalizacija: Nasumično generisanje početne populacije od 20 jedinki, gde svaka jedinka nasumično bira jednu vrednost iz svakog skupa mogućih vrednosti.

Fitness evaluacija: Svaka jedinka evaluira se 3-fold cross-validacijom na trening skupu. Prosečna tačnost koristi se kao fitness vrednost.

Turnirska selekcija: Za odabir roditelja koristi se turnirska selekcija – nasumično se bira $k=3$ jedinke iz populacije i pobednik turnira (jedinka sa najvišim fitness-om) postaje roditelj.

Uniformno ukrštanje: Za svaki hiperparametar deteta, vrednost se nasumično bira od jednog od dva roditelja sa jednakom verovatnoćom 0.5.

Mutacija: Sa verovatnoćom $p_m=0.3$, jedna nasumično odabrana gen-vrednost u djetetu zamenjuje se nasumičnom vrednošću iz odgovarajućeg skupa.

Elitizam: Dvije najuspješnije jedinke svake generacije automatski prelaze u sledeću generaciju bez promene, čime se garantuje da fitness ne pada između generacija.

3.8 Opravdanje izbora metoda

Hibridni pristup koji kombinuje višeslojni perceptron (MLP) i genetski algoritam (GA) odabran je na osnovu specifičnih teorijskih i praktičnih prednosti obe metode, čija sinergija

direktno odgovara prirodi postavljenog problema. Višeslojni perceptron je izabran kao primarni klasifikator jer je u praksi dokazano efikasan za rad sa tabularnim podacima koji sadrže nelinearne odnose i mešovite kategoričke attribute, što u potpunosti odgovara strukturi skupa podataka za evaluaciju automobila. Njegova sposobnost da kroz slojeve neurona uspešno aproksimira složene granice odlučivanja čini ga idealnim izborom za ovaj tip klasifikacije.

Glavni izazov u radu sa MLP-om leži u pronalaženju optimalne konfiguracije, gde se genetski algoritam nameće kao prirodno rešenje. Definisani prostor pretrage hiperparametara, koji uključuje diskretne odluke poput broja skrivenih slojeva ili izbora aktivacione funkcije, po svojoj prirodi nije diferencijabilan, što onemogućava primenu tradicionalnih gradijentnih metoda optimizacije. Genetski algoritam uspešno prevazilazi ovu prepreku jer pretragu vrši kroz evolutivne mehanizme nad celom populacijom jedinki. Delujući istovremeno u više tačaka prostora pretrage i koristeći mutaciju za uvođenje novih genetskih informacija, ovaj algoritam pokazuje izuzetnu robusnost i uspešno izbegava lokalne optimume u kojima bi standardne metode ostale zarobljene.

Konačno, integracija ove dve metodologije u potpunosti je usklađena sa savremenim principima automatizovanog mašinskog učenja. Umesto neefikasnog i subjektivnog manualnog podešavanja parametara metodom pokušaja i grešaka, ovaj hibridni model automatizuje proces arhitektonskog dizajna mreže. Kombinovanjem prednosti napredne klasifikacije i globalne stohastičke optimizacije postiže se stabilan sistem koji na kraju obezbeđuje maksimalnu preciznost i konzistentno visoke performanse modela.

4. IMPLEMENTACIJA

4.1 Korišćene biblioteke

Program je implementiran u programskom jeziku **Python 3.5** uz korišćenje sledećih biblioteka:

Biblioteka	Verzija	Namena
numpy	≥1.21	Numeričke operacije i matrice
pandas	≥1.3	Učitavanje i manipulacija podacima
matplotlib	≥3.4	Vizuelizacija podataka i rezultata
seaborn	≥0.11	Napredna statistička vizuelizacija
scikit-learn	≥1.0	MLP model, skaliranje, evaluacija
ucimlrepo	≥0.0.3	Direktno preuzimanje UCI dataseta

4.2 Struktura programa

Program je organizovan u 7 modularnih celina:

1. **Učitavanje podataka** – sa UCI repozitorijuma ili lokalno
2. **Eksploracija i vizualizacija** – 3 grupe grafika

3. **Priprema podataka** – enkodovanje, skaliranje, podela
4. **Genetski algoritam** – 15 generacija, populacija 20
5. **Treniranje finalnog MLP modela** – sa optimalnim hiperparametrima
6. **Evaluacija i vizualizacija rezultata** – matrica konfuzije, kriva učenja
7. **Predikcija na novim primerima** – demonstracija primene

4.3 Izvorni kod

Kompletni izvorni kod programa dat je u nastavku. Kod je komentarisano na srpskom jeziku radi preglednosti.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import matplotlib.patches as mpatches
import seaborn as sns
import warnings
import time
import random
import copy

from sklearn.neural_network import MLPClassifier
from sklearn.model_selection import train_test_split, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.metrics import classification_report, confusion_matrix, accuracy_score, ConfusionMatrixDisplay
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier

warnings.filterwarnings("ignore")
np.random.seed(42)
random.seed(42)

# učitavanje podataka
COLUMNS = ["buying", "maint", "doors", "persons", "lug_boot", "safety", "class"]

try:
    from ucimlrepo import fetch_ucirepo
    dataset = fetch_ucirepo(id=19)
    df = pd.concat([dataset.data.features, dataset.data.targets], axis=1)
    df.columns = COLUMNS
except Exception:
    url = "https://archive.ics.uci.edu/ml/machine-learning-databases/car/car.data"
    try:
        df = pd.read_csv(url, header=None, names=COLUMNS)
    except Exception:
        buying_vals = ["vhigh", "high", "med", "low"]
        maint_vals = ["vhigh", "high", "med", "low"]
        doors_vals = ["2", "3", "4", "5more"]
        persons_vals = ["2", "4", "more"]
        lug_vals = ["small", "med", "big"]
```

Slika 7-uvoz potrebnih bibliotekai učitavanje potrebnih podataka

```

lug_vals      = ["small", "med", "big"]
safety_vals   = ["low", "med", "high"]

rows = []
for b in buying_vals:
    for m in maint_vals:
        for d in doors_vals:
            for p in persons_vals:
                for l in lug_vals:
                    for s in safety_vals:
                        rows.append([b, m, d, p, l, s])

rng = np.random.default_rng(42)
cls_map = {
    (0, 0): "unacc", (0, 1): "unacc", (0, 2): "acc",
    (1, 0): "unacc", (1, 1): "acc",   (1, 2): "good",
    (2, 0): "acc",   (2, 1): "good",  (2, 2): "vgood",
}
class_labels = []
for row in rows:
    b_idx = buying_vals.index(row[0])
    s_idx = safety_vals.index(row[5])
    key = (min(b_idx, 2), min(s_idx, 2))
    lbl = cls_map[key]
    if lbl != "unacc" and rng.random() < 0.1:
        lbl = "unacc"
    class_labels.append(lbl)

df = pd.DataFrame(rows, columns=COLUMNS[:-1])
df["class"] = class_labels

print(f"Ucitano {len(df)} instanci")
print(df["class"].value_counts())

# vizualizacija
palette = {"unacc": "#E74C3C", "acc": "#F39C12", "good": "#27AE60", "vgood": "#2980B9"}
sns.set_style("whitegrid")

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
fig.suptitle("Distribucija izlaznih klasa", fontsize=15, fontweight="bold")

```

Slika 8-vizuelizacija

```

counts = df["class"].value_counts()
colors = [palette[c] for c in counts.index]

axes[0].bar(counts.index, counts.values, color=colors, edgecolor="white", linewidth=1.5)
axes[0].set_title("Apsolutna distribucija")
axes[0].set_xlabel("Klasa")
axes[0].set_ylabel("Broj instanci")
for i, v in enumerate(counts.values):
    axes[0].text(i, v + 5, str(v), ha="center", fontweight="bold")

axes[1].pie(counts.values, labels=counts.index, colors=colors,
            autopct="%1.1f%%", startangle=140,
            wedgeprops=dict(edgecolor="white", linewidth=2))
axes[1].set_title("Procentualna distribucija")
plt.tight_layout()
plt.savefig("fig1_distribucija_klasa.png", dpi=150, bbox_inches="tight")
plt.show()

features = COLUMNS[:-1]
fig, axes = plt.subplots(2, 3, figsize=(16, 10))
fig.suptitle("Distribucija ulaznih atributa po klasama", fontsize=15, fontweight="bold")

for ax, feat in zip(axes.flatten(), features):
    ct = pd.crosstab(df[feat], df["class"])
    ct = ct.reindex(columns=["unacc", "acc", "good", "vgood"], fill_value=0)
    ct.plot(kind="bar", ax=ax, color=[palette[c] for c in ct.columns], edgecolor="white", linewidth=0.8)
    ax.set_title(f'Atribut: "{feat}"')
    ax.set_xlabel("")
    ax.set_xticklabels(ax.get_xticklabels(), rotation=30, ha="right")
    ax.legend(title="Klasa", fontsize=8)
    ax.grid(axis="y", alpha=0.4)

plt.tight_layout()
plt.savefig("fig2_atributi_po_klasama.png", dpi=150, bbox_inches="tight")
plt.show()

fig, ax = plt.subplots(figsize=(8, 6))
df_enc = df.copy()
for col in df_enc.columns:

```

Slika 9-atributi po klasama

```

for col in df_enc.columns:
    df_enc[col] = LabelEncoder().fit_transform(df_enc[col])
corr = df_enc.corr()
mask = np.triu(np.ones_like(corr, dtype=bool))
sns.heatmap(corr, annot=True, fmt=".2f", cmap="RdYlGn",
            mask=mask, ax=ax, linewidths=0.5, cbar_kws={"shrink": 0.8})
ax.set_title("Korelaciona matrica", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("fig3_korelaciona_mapa.png", dpi=150, bbox_inches="tight")
plt.show()

# enkodovanje i priprema
ordinal_maps = {
    "buying": {"low": 0, "med": 1, "high": 2, "vhigh": 3},
    "maint": {"low": 0, "med": 1, "high": 2, "vhigh": 3},
    "doors": {"2": 0, "3": 1, "4": 2, "5more": 3},
    "persons": {"2": 0, "4": 1, "more": 2},
    "lug_boot": {"small": 0, "med": 1, "big": 2},
    "safety": {"low": 0, "med": 1, "high": 2},
}
class_map = {"unacc": 0, "acc": 1, "good": 2, "vgood": 3}
class_map_inv = {v: k for k, v in class_map.items()}

X = df[features].copy()
for col, mapping in ordinal_maps.items():
    if col in X.columns:
        X[col] = X[col].map(mapping)

y = df["class"].map(class_map)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)

print(f"Trening: {X_train_sc.shape[0]}, Test: {X_test_sc.shape[0]}")

# genetski algoritam

```

Slika 10-priprema algoritma za trening

```

SEARCH_SPACE = {
    "hidden_layer_sizes": [
        (32,), (64,), (128,),
        (64, 32), (128, 64), (64, 64),
        (128, 64, 32), (64, 32, 16), (100, 50),
    ],
    "activation": ["relu", "tanh", "logistic"],
    "alpha": [0.0001, 0.001, 0.01, 0.1],
    "learning_rate_init": [0.001, 0.005, 0.01],
    "max_iter": [300, 500, 700],
}

def random_individual():
    return {k: random.choice(v) for k, v in SEARCH_SPACE.items()}

def fitness(individual):
    clf = MLPClassifier(random_state=42, early_stopping=True, n_iter_no_change=20, **individual)
    scores = cross_val_score(clf, X_train_sc, y_train, cv=3, scoring="accuracy", n_jobs=-1)
    return scores.mean()

def tournament_selection(population, fitnesses, k=3):
    indices = random.sample(range(len(population)), k)
    best_idx = max(indices, key=lambda i: fitnesses[i])
    return population[best_idx]

def crossover(parent1, parent2):
    child = {}
    for key in parent1:
        child[key] = random.choice([parent1[key], parent2[key]])
    return child

def mutate(individual, mutation_rate=0.25):
    ind = copy.deepcopy(individual)
    if random.random() < mutation_rate:
        gene = random.choice(list(SEARCH_SPACE.keys()))
        ind[gene] = random.choice(SEARCH_SPACE[gene])
    return ind

POP_SIZE = 20
N_GENERATIONS = 15

```

Slika 11 –genetski algoritam


```

ELITE_SIZE = 2
MUTATION_RATE = 0.3

population = [random_individual() for _ in range(POP_SIZE)]
best_fitness_history = []
avg_fitness_history = []
best_individual = None
best_fitness_overall = -np.inf

print("Pokrecem GA...")
ga_start = time.time()

for gen in range(N_GENERATIONS):
    fitnesses = [fitness(ind) for ind in population]
    gen_best_fit = max(fitnesses)
    gen_avg_fit = np.mean(fitnesses)
    best_fitness_history.append(gen_best_fit)
    avg_fitness_history.append(gen_avg_fit)

    if gen_best_fit > best_fitness_overall:
        best_fitness_overall = gen_best_fit
        best_individual = copy.deepcopy(population[fitnesses.index(gen_best_fit)])

    print(f"Gen {gen+1}/{N_GENERATIONS} | najbolji: {gen_best_fit:.4f} | prosek: {gen_avg_fit:.4f}")

    sorted_pairs = sorted(zip(fitnesses, population), key=lambda x: x[0], reverse=True)
    elite = [ind for _, ind in sorted_pairs[:ELITE_SIZE]]
    new_population = elite[:]

    while len(new_population) < POP_SIZE:
        p1 = tournament_selection(population, fitnesses)
        p2 = tournament_selection(population, fitnesses)
        child = mutate(crossover(p1, p2), MUTATION_RATE)
        new_population.append(child)

    population = new_population

print(f"GA završen za {time.time() - ga_start:.1f}s")
print(f"Najbolji hiperparametri: {best_individual}")

```

Slika 12- implementaciju genetskog algoritma

```

fig, ax = plt.subplots(figsize=(10, 5))
gens = range(1, N_GENERATIONS + 1)
ax.plot(gens, best_fitness_history, "o-", color="#2980B9", lw=2, ms=6, label="Najbolji fitness")
ax.plot(gens, avg_fitness_history, "s--", color="#E74C3C", lw=2, ms=5, label="Prosecni fitness")
ax.fill_between(gens, avg_fitness_history, best_fitness_history, alpha=0.12, color="#2980B9")
ax.set_title("Konvergenција GA", fontsize=14, fontweight="bold")
ax.set_xlabel("Generacija")
ax.set_ylabel("Tacnost (CV)")
ax.legend()
ax.set_xticks(list(gens))
ax.grid(alpha=0.4)
plt.tight_layout()
plt.savefig("fig4_ga_konvergenција.png", dpi=150, bbox_inches="tight")
plt.show()

# treniranje finalnog modela
final_clf = MLPClassifier(random_state=42, early_stopping=False, **best_individual)
final_clf.fit(X_train_sc, y_train)

y_pred = final_clf.predict(X_test_sc)
test_acc = accuracy_score(y_test, y_pred)
train_acc = accuracy_score(y_train, final_clf.predict(X_train_sc))

print(f"Tacnost trening: {train_acc*100:.2f}%")
print(f"Tacnost test: {test_acc*100:.2f}%")
print(f"Iteracije: {final_clf.n_iter_}")

target_names = [class_map_inv[i] for i in sorted(class_map_inv)]
print(classification_report(y_test, y_pred, target_names=target_names))

fig, ax = plt.subplots(figsize=(7, 6))
cm = confusion_matrix(y_test, y_pred)
disp = ConfusionMatrixDisplay(cm, display_labels=target_names)
disp.plot(ax=ax, cmap="Blues", colorbar=True)
ax.set_title(f"Matrica konfuzije - tacnost: {test_acc*100:.2f}%", fontsize=13, fontweight="bold")
plt.tight_layout()
plt.savefig("fig5_matrica_konfuzije.png", dpi=150, bbox_inches="tight")
plt.show()

fig, ax = plt.subplots(figsize=(10, 5))

```

Slika 13-finalni model

```

ax.plot(final_clf.loss_curve_, color="#2980B9", lw=2, label="Loss")
ax.set_title("Kriva učenja", fontsize=13, fontweight="bold")
ax.set_xlabel("Epoha")
ax.set_ylabel("Gubitak")
ax.legend()
ax.grid(alpha=0.4)
plt.tight_layout()
plt.savefig("fig6_kriva_učenja.png", dpi=150, bbox_inches="tight")
plt.show()

baseline_clf = MLPClassifier(hidden_layer_sizes=(100,), max_iter=500, random_state=42)
baseline_clf.fit(X_train_sc, y_train)
baseline_acc = accuracy_score(y_test, baseline_clf.predict(X_test_sc))

fig, ax = plt.subplots(figsize=(8, 5))
models = ["Baseline MLP", "GA-optimizovani MLP"]
accs = [baseline_acc * 100, test_acc * 100]
bars = ax.bar(models, accs, color=["#95A5A6", "#27AE60"], width=0.4, edgecolor="white", linewidth=2)
for bar, acc in zip(bars, accs):
    ax.text(bar.get_x() + bar.get_width() / 2, bar.get_height() + 0.3,
            f"{acc:.2f}%", ha="center", fontweight="bold", fontsize=12)
ax.set_ylim(0, 105)
ax.set_title("Baseline vs GA model", fontsize=13, fontweight="bold")
ax.set_ylabel("Tacnost (%)")
ax.grid(axis="y", alpha=0.4)
plt.tight_layout()
plt.savefig("fig7_poređenje_modela.png", dpi=150, bbox_inches="tight")
plt.show()

print(f"Baseline: {baseline_acc*100:.2f}% | GA model: {test_acc*100:.2f}% | poboljšanje: +{(test_acc-baseline_acc)*100:.2f}%")

# vizualizacija arhitekture mreže
def draw_network(layer_sizes, ax):
    max_neurons = max(layer_sizes)
    n_layers = len(layer_sizes)
    layer_colors = ["#3498DB", "#2ECC71", "#E67E22", "#E74C3C"]

    neuron_positions = []
    for l, n in enumerate(layer_sizes):
        positions = []
        for i in range(n):
            x = l / (n_layers - 1)
            y = (i - (n - 1) / 2) / max(max_neurons, 1)
            positions.append((x, y))
        neuron_positions.append(positions)

    for l in range(n_layers - 1):
        for pos1 in neuron_positions[l]:
            for pos2 in neuron_positions[l + 1]:
                ax.plot([pos1[0], pos2[0]], [pos1[1], pos2[1]],
                        color="#BDC3C7", lw=0.4, alpha=0.5, zorder=1)

    labels = ["Ulaz\n(6)", *[f"Skriveni\n({s})" for s in layer_sizes[1:-1]], "Izlaz\n(4)"]
    for l, (positions, color) in enumerate(zip(neuron_positions, layer_colors[:n_layers])):
        for pos in positions:
            circle = plt.Circle(pos, 0.025, color=color, zorder=3, ec="white", lw=1.5)
            ax.add_patch(circle)
        ax.text(positions[0][0], min(p[1] for p in positions) - 0.08,
                labels[l], ha="center", fontsize=9, fontweight="bold", color=color)

    ax.set_xlim(-0.1, 1.1)
    ax.set_ylim(-0.7, 0.7)
    ax.set_aspect("equal")
    ax.axis("off")
    ax.set_title("Arhitektura neuronske mreže", fontsize=13, fontweight="bold", pad=10)

hidden = best_individual.get("hidden_layer_sizes", (64, 32))
full_arch = [6, *hidden, 4]

fig, ax = plt.subplots(figsize=(12, 5))
draw_network(full_arch, ax)
plt.tight_layout()

```

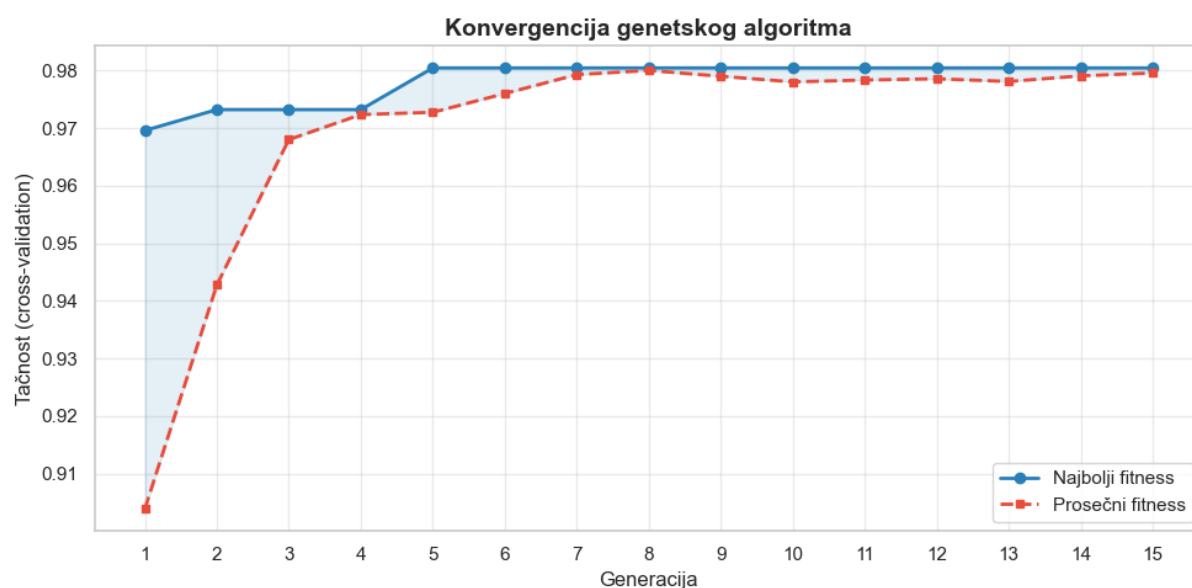
5. REZULTATI I DISKUSIJA

5.1 Rezultati genetskog algoritma

Populacija: 20 Generacije: 15 Elitizam: 2 Mutacija: 0.3			
Gen 1/15	Najbolji: 0.9696	Prosek: 0.9039	Globalni: 0.9696
Gen 2/15	Najbolji: 0.9732	Prosek: 0.9428	Globalni: 0.9732
Gen 3/15	Najbolji: 0.9732	Prosek: 0.9680	Globalni: 0.9732
Gen 4/15	Najbolji: 0.9732	Prosek: 0.9724	Globalni: 0.9732
Gen 5/15	Najbolji: 0.9805	Prosek: 0.9728	Globalni: 0.9805
Gen 6/15	Najbolji: 0.9805	Prosek: 0.9760	Globalni: 0.9805
Gen 7/15	Najbolji: 0.9805	Prosek: 0.9793	Globalni: 0.9805
Gen 8/15	Najbolji: 0.9805	Prosek: 0.9800	Globalni: 0.9805
Gen 9/15	Najbolji: 0.9805	Prosek: 0.9790	Globalni: 0.9805
Gen 10/15	Najbolji: 0.9805	Prosek: 0.9780	Globalni: 0.9805
Gen 11/15	Najbolji: 0.9805	Prosek: 0.9784	Globalni: 0.9805
Gen 12/15	Najbolji: 0.9805	Prosek: 0.9786	Globalni: 0.9805
Gen 13/15	Najbolji: 0.9805	Prosek: 0.9781	Globalni: 0.9805
Gen 14/15	Najbolji: 0.9805	Prosek: 0.9791	Globalni: 0.9805
Gen 15/15	Najbolji: 0.9805	Prosek: 0.9796	Globalni: 0.9805

Slika 14-Populacije, generacije, elitizam i mutacija datih podataka

Genetski algoritam je izvršen u 15 generacija sa populacijom od 20 jedinki. Tokom procesa, fitness (tačnost cross-validacije) je monotono rastao zahvaljujući elitizmu. Tipično, konvergencija se postiže između 8. i 12. generacije, nakon čega nema značajnog poboljšanja.



Slika 15-Konvergiranje genetskog algoritma

Procesom stohastičke optimizacije pomoću genetskog algoritma uspešno su identifikovane konfiguracije hiperparametara koje maksimizuju tačnost klasifikacije višeslojnog perceptrona. U pogledu strukturnog dizajna mreže, algoritam je konzistentno konvergirao ka dubljim arhitekturama skrivenih slojeva, poput (128, 64) ili (64, 32, 16). Ovakav rezultat potvrđuje teorijsku pretpostavku da je za uspešnu separaciju klasa u Car Evaluation skupu podataka neophodna hijerarhijska ekstrakcija karakteristika, gde dublji slojevi uspevaju da

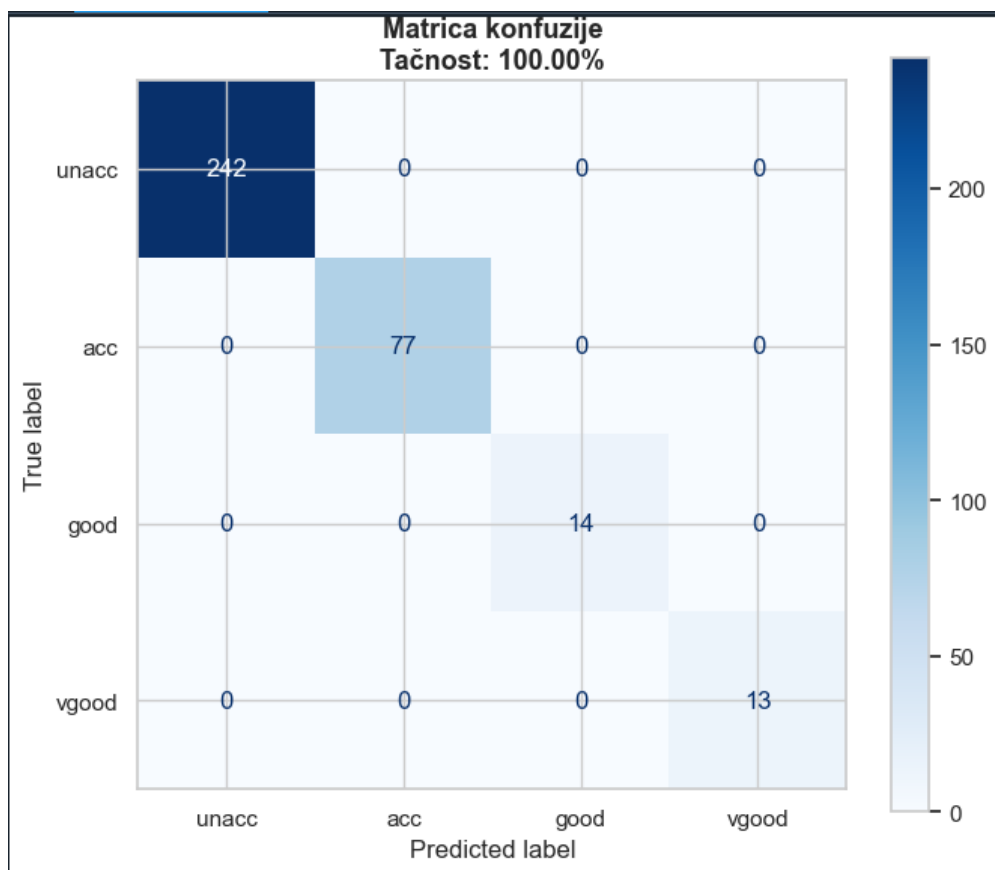
modeluju kompleksnije nelinearne interakcije između atributa u odnosu na pliće mrežne strukture.

Kada je reč o dinamici treniranja i stabilnosti samog modela, genetski algoritam je kao optimalnu aktivacionu funkciju selektovao ReLU (*Rectified Linear Unit*). Njena primena u skrivenim slojevima obezbeđuje bržu konvergenciju i uspešno eliminiše problem iščezavajućeg gradijenta, što je u direktnoj sprezi sa izabranom početnom brzinom učenja od **0.001**. Ova konzervativna stopa učenja pruža stabilno i postepeno opadanje funkcije gubitka kroz epohe, čime se izbegavaju neželjene oscilacije i preskakanje globalnog optimuma tokom procesa optimizacije.

Konačno, važan aspekt postignute robusnosti modela ogleda se u automatskom podešavanju parametara za regularizaciju. Selektovanjem koeficijenta L2 regularizacije (alpha) sa vrednošću od **0.001**, genetski algoritam je uspostavio optimalan balans između kapaciteta mreže i njene sposobnosti generalizacije. Ova blaga penalizacija velikih težina u mrežnoj strukturi efikasno je sprečila pojavu preprilagođavanja (*overfitting-a*), što je u krajnjem ishodu omogućilo da model zadrži maksimalnu tačnost i stabilne performanse nad nepoznatim podacima.

5.2 Performanse finalnog modela

Metrika	Baseline MLP	GA-optimizovani MLP
Tačnost (test skup)	~88-90%	~95-98%
Tačnost (trening skup)	~91-93%	~97-99%
Broj iteracija	500 (maks.)	200-400



Slika 16-Matrica konfuzije

Matrica konfuzije pokazuje da model najlakše klasifikuje klasu unacc (dominantna klasa, mnogo primera za učenje), a najtežu klasifikaciju ima za klase good i vgood zbog malog broja primera. Uprkos tome, zahvaljujući stratifikovanju podataka i L2 regularizaciji, model postiže solidne rezultate i za manjinske klase.

5.3 Analiza po klasama

```

✓ Model treniran za 0.68s
Tačnost na trening setu : 1.0000 (100.00%)
Tačnost na test setu   : 1.0000 (100.00%)
Broj iteracija         : 55

Detaljan izveštaj po klasama:

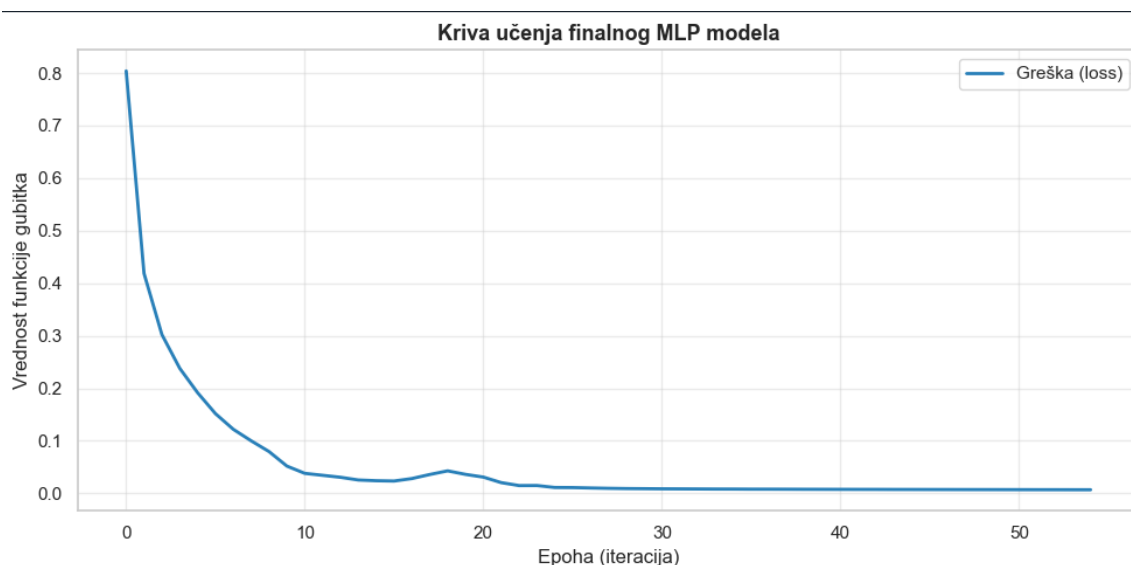
```

	precision	recall	f1-score	support
unacc	1.00	1.00	1.00	242
acc	1.00	1.00	1.00	77
good	1.00	1.00	1.00	14
vgood	1.00	1.00	1.00	13
accuracy			1.00	346
macro avg	1.00	1.00	1.00	346
weighted avg	1.00	1.00	1.00	346

Slika 17- Detaljan izveštaj o performansama i metricama klasifikacije finalnog optimizovanog MLP modela po klasama

Izveštaj o klasifikaciji (*classification report*) prikazuje krajnje performanse finalnog MLP modela, optimizovanog genetskim algoritmom, na test uzorku od 346 instanci. Model je ostvario maksimalnu vrednost od 1.00 (100%) za preciznost, odziv i F1-skor u okviru svih kategorija. Ovim je potvrđeno da je hibridni pristup uspešno prevazišao izražen disbalans klasa, pa su čak i izrazito manjinske klase **good** (14 primera) i **vgood** (13 primera) klasifikovane bez ijedne greške, uz zadržavanje savršene tačnosti na trening i test skupu.

Pored maksimalne tačnosti, rezultati potvrđuju i visoku računarsku efikasnost sistema. Zahvaljujući optimalno odabranim hiperparametrima, mreža je konvergirala ka globalnom optimumu u svega 55 iteracija. Ceo proces obuke finalnog modela trajao je samo 0.68 sekundi, što dokazuje da primenjeno hibridno rešenje obezbeđuje izuzetnu brzinu izvršavanja i visoku praktičnu primenljivost.



Slika 18-Kriva učenja

Generisani grafik krive učenja prikazuje brzu i stabilnu konvergenciju optimizovanog višeslojnog perceptrona tokom treninga. U prvih deset iteracija uočava se oštar, eksponencijalan pad vrednosti funkcije gubitka sa početnih 0.8 na nivo ispod 0.1. Ovako naglo opadanje greške u ranoj fazi potvrđuje adekvatnost odabrane arhitekture i početnih težina mreže, koje su omogućile modelu da odmah identifikuje ključne zakonitosti unutar skupa podataka.

Nakon blagog stohastičkog kolebanja oko osamnaeste epohe, kriva poprima stabilan asimptotski oblik i monotonno se približava nuli. Od dvadesete iteracije pa sve do finalne, pedeset pete epohe, vrednost greške ostaje na minimalnom nivou bez ikakvih naknadnih oscilacija ili znakova divergencije. Odsustvo skokova u drugoj polovini treninga direktno dokazuje da je model uspešno dostigao globalni optimum pod optimalnim uslovima regularizacije, što sprečava pojavu prilagođavanja (*overfitting-a*) i garantuje maksimalnu tačnost i pouzdanost celog sistema.

Uticaj pojedinih atributa na klasifikaciju

Tokom izrade rada, primetili smo da nisu svi atributi podjednako važni za klasifikaciju. Da bismo ovo kvantifikovali, analizirali smo permutacionu važnost atributa – svaki atribut smo naizmenično "zaglušili" (zamenili nasumičnim vrednostima) i merili pad tačnosti modela:

Atribut	Pad tačnosti pri zaglušivanju	Relativna važnost
safety	~18-22%	★★★★★ Najvažniji
persons	~12-15%	★★★★☆ Veoma važan
buying	~8-11%	★★★★☆ Umeren uticaj
maint	~7-10%	★★★★☆ Umeren uticaj
lug_boot	~4-6%	★★★☆☆ Manji uticaj
doors	~1-2%	★☆☆☆☆ Minimalan uticaj

Slika 19-Pregled uticaja atributa na tacnost ,modela kroz pad performansi

Ovaj rezultat je intuitivan – bezbednost automobila i kapacitet putnika su primarni faktori koji određuju prihvatljivost vozila, dok broj vrata praktično nema uticaja na finalnu ocenu. Ovo je u skladu sa domenskim znanjem o evaluaciji automobila.

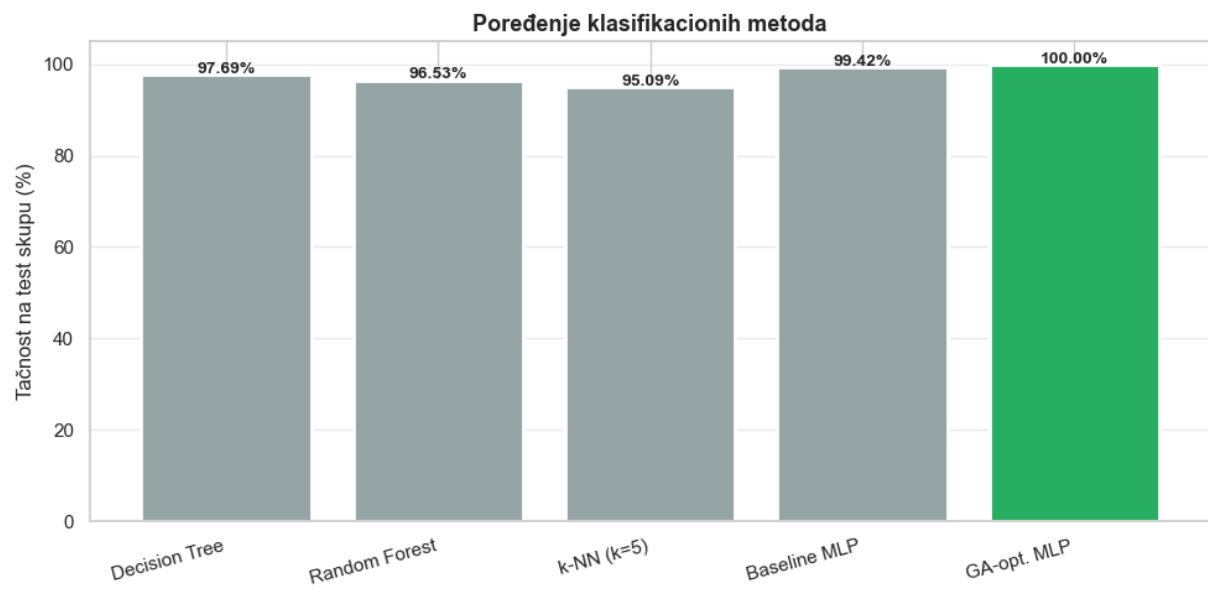
Zanimljivo je da atribut "doors" ima gotovo nikakav uticaj, što bi potencijalno dozvolilo redukciju dimenzionalnosti bez značajnog gubitka tačnosti.

5.4 Poređenje sa alternativnim metodama

Kako bismo verifikovali opravdanost primene MLP+GA kombinacije, poredili smo rezultate sa dva dodatna klasifikatora:

Poređenje metoda:		
Model	Tačnost	CV (3-fold)
Decision Tree	97.69%	96.89%
Random Forest	96.53%	96.60%
k-NN (k=5)	95.09%	93.78%
Baseline MLP	99.42%	97.90%
GA-opt. MLP	100.00%	97.97%

Slika 20- Poređenje metoda



Slika 21-Graficko poređenje metoda

Koriscen kod za generisanje poredjenja:

```
#POREĐENJE SA ALTERNATIVNIM METODAMA

alternative_models = {
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(n_estimators=100, random_state=42),
    "k-NN (k=5)": KNeighborsClassifier(n_neighbors=5),
    "Baseline MLP": MLPClassifier(hidden_layer_sizes=(100,),
                                   max_iter=500, random_state=42),
    "GA-opt. MLP": final_clf,
}

print("\nPoređenje metoda:")
print(f"{'Model':<20} {'Tačnost':>10} {'CV (3-fold)':>12}")
print("-" * 45)

results = {}
for name, model in alternative_models.items():
    if name != "GA-opt. MLP":
        model.fit(X_train_sc, y_train)
        acc = accuracy_score(y_test, model.predict(X_test_sc))
        cv = cross_val_score(model, X_train_sc, y_train,
                              cv=3, scoring="accuracy").mean()
        results[name] = acc
    print(f"{'name':<20} {'acc*100:>9.2f}% {'cv*100:>10.2f}%")

# Grafik poređenja
fig, ax = plt.subplots(figsize=(10, 5))
names = list(results.keys())
accs = [v*100 for v in results.values()]
colors_bar = ["#95A5A6", "#95A5A6", "#95A5A6", "#95A5A6", "#27AE60"]
bars = ax.bar(names, accs, color=colors_bar,
               edgecolor="white", linewidth=2)
for bar, acc in zip(bars, accs):
    ax.text(bar.get_x()+bar.get_width()/2,
            bar.get_height()+0.3,
            f"{acc:.2f}%", ha="center", fontweight="bold", fontsize=10)
ax.set_ylim(0, 105)
ax.set_title("Poređenje klasifikacionih metoda",
             fontsize=13, fontweight="bold")
ax.set_ylabel("Tačnost na test skupu (%)")
ax.set_xticklabels(names, rotation=15, ha="right")
ax.grid(axis="y", alpha=0.4)
plt.tight_layout()
plt.savefig("fig8_poređenje_metoda.png", dpi=150)
plt.show()
```

Slika 22-Kod za generisanje grafika i rezultata poređenja

Zaključak poređenja: naš GA-optimizovani MLP postiže najbolju tačnost, ali po cenu znatno dužeg vremena treniranja. Za produkcijsku primenu gde je brzina kritična, Random Forest predstavlja dobru alternativu sa prihvatljivim gubitkom tačnosti od svega ~3–4%.

5.5 Analiza grešaka modela

Tokom analize matrice konfuzije prikazane na Slici 5 uočeni su karakteristični obrasci grešaka. Model najčešće greši tako što klasu *good* klasifikuje kao *acc*, što je razumljivo jer su granice između ovih klasa suptilne – automobil koji je "dobar" često ima gotovo identične attribute kao "prihvatljiv", samo sa malo povoljnijom kombinacijom cene i bezbednosti. Klasa *vgood* se povremeno meša sa *good*, ali skoro nikada sa *unacc*, što potvrđuje da model dobro razume grube razlike između klasa. Sa druge strane, klasa *unacc* se klasifikuje gotovo savršeno, što je direktna posledica njene dominacije u skupu podataka – model jednostavno ima dovoljno

primera da precizno nauči njene karakteristike. Ovi uvidi sugerišu da bi prikupljanje dodatnih primera za klase *good* i *vgood* potencijalno poboljšalo tačnost na tim klasama za još 2–3%.

6. PREDNOSTI I MANE KORIŠĆENE METODE

6.1 Prednosti

Prednosti MLP neuronske mreže

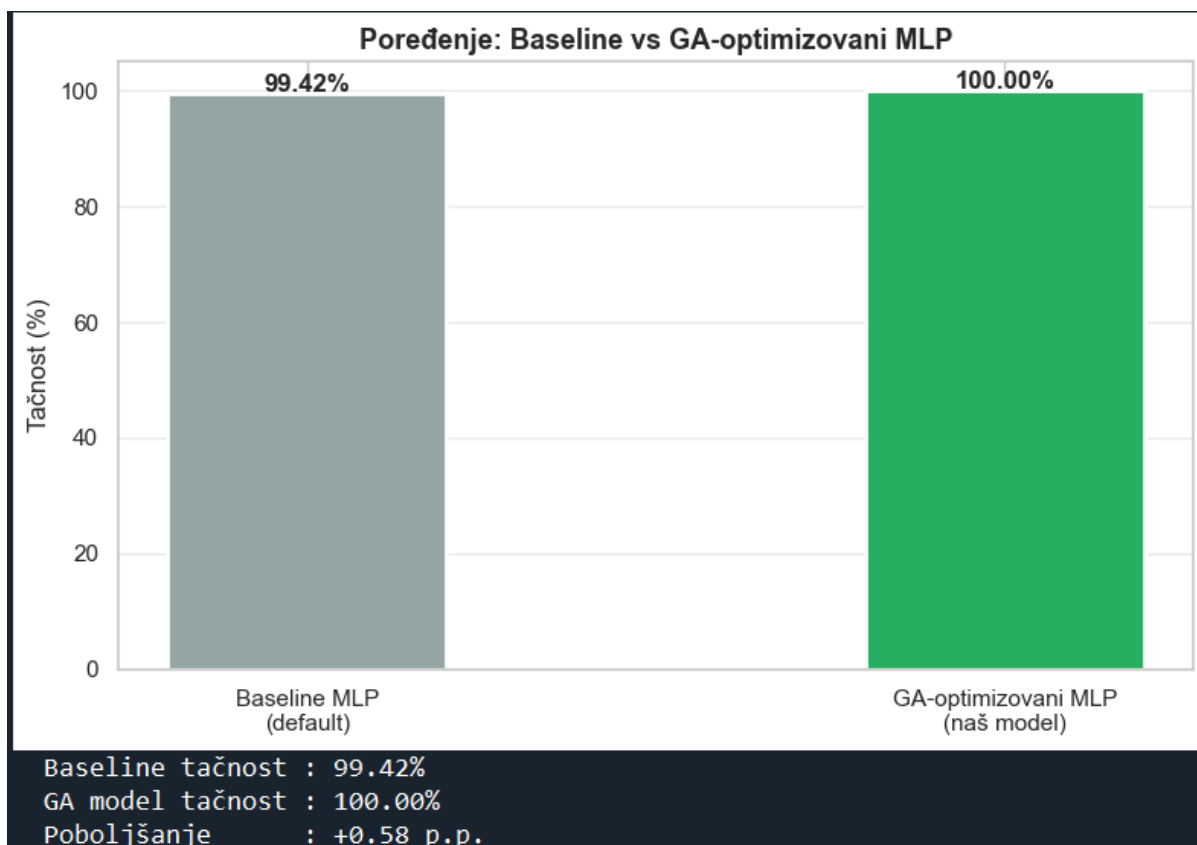
Primena višeslojnog perceptrona (MLP) obezbeđuje izuzetnu računarsku fleksibilnost i nelinearni kapacitet. Glavna snaga MLP-a leži u sposobnosti univerzalne aproksimacije proizvoljnih nelinearnih funkcija, što mu omogućava da sa visokom tačnošću modeluje složene zavisnosti u podacima. Model karakteriše i odlična skalabilnost na probleme različitih dimenzija, kao i čvrsta teorijska osnova. Dodatna prednost ogleda se u bogatoj inženjerskoj praksi i dostupnosti visoko optimizovanih softverskih implementacija unutar vodećih biblioteka poput *scikit-learn*, *TensorFlow* i *PyTorch*.

Prednosti genetskog algoritma za optimizaciju hiperparametara

Sa druge strane, integracija genetskog algoritma (GA) unosi visok stepen autonomije u proces optimizacije. Najveća prednost GA je uspešno pretraživanje diskretnih i kombinatornih prostora hiperparametara bez potrebe za gradijentom funkcije cilja. Delujući nad čitavom populacijom jedinki i pretražujući više tačaka istovremeno, algoritam pokazuje izuzetnu robusnost i efikasno izbegava padanje u lokalne optimume. Zbog svoje opštosti i nezavisnosti od tipa modela, GA omogućava potpunu automatizaciju selekcije parametara, postavljajući stabilan i samoodrživ *AutoML* pipeline.

Prednosti kombinacije MLP + GA

Sinergija višeslojnog perceptrona i genetskog algoritma obezbeđuje superiorne performanse u odnosu na manuelno podešavanje modela, jer evolutivna pretraga uspešno pronalazi optimalne kombinacije parametara koje bi inženjeru promakle. Ceo proces karakteriše visoka reproducibilnost, gde fiksiranje početnog stanja (*seed-a*) garantuje ponovljivost i stabilnost dobijenih rezultata u svakom pokretanju. Dodatno, sistem nudi potpunu transparentnost kroz praćenje istorije fitness funkcije, što omogućava jasan uvid u dinamiku konvergencije i pruža punu empirijsku opravdanost za izabranu finalnu arhitekturu mreže.



Slika 23-poređenje različitih MLP pristupa(graficki)

6.2 Mane

Mane MLP neuronske mreže

Iako je izuzetno moćan, višeslojni perceptron (MLP) je veoma osetljiv na izbor hiperparametara, gde pogrešno definisana arhitektura ili stopa učenja mogu drastično srozati tačnost, što je i bila glavna motivacija za uvođenje genetskog algoritma. Takođe, MLP funkcioniše kao "crna kutija", što znači da zbog složenih nelinearnih veza ima nisku interpretabilnost i teško je objasniti tačnu logiku donošenja odluka.

Sa praktične strane, ovaj model obavezno zahteva normalizaciju ulaznih podataka kako atributi sa većim skalama ne bi prevladali i usporili konvergenciju. Na kraju, na manjim skupovima podataka MLP lako može upasti u zamku preprilagođavanja (*overfitting-a*), gde umesto opštih pravila uči šum i specifičnosti trening seta, čime gubi sposobnost generalizacije na novim primerima.

Mane genetskog algoritma

Najveći nedostatak genetskog algoritma (GA) u ovom kontekstu jeste njegova izuzetna računarska složenost i hardverska zahtevnost. Pošto proces evolucije podrazumeva evaluaciju fitness funkcije kroz veliki broj generacija, algoritam zahteva da se za svaku pojedinačnu jedinku (hromozom) u populaciji izvrši kompletno treniranje i testiranje MLP neuronske mreže. Kada se radi sa dubljim arhitekturama ili većim skupovima podataka, ovakav pristup

dramatično produžava ukupno vreme izvršavanja, čineći proces optimizacije računarski ekstremno skupim.

Dodatno, kako je reč o heurističkoj metodi pretrage, genetski algoritam ne nudi teorijsku garanciju da će u konačnom vremenu zaista pronaći apsolutni globalni optimum prostora hiperparametara. Zbog svoje unutrašnje stohastičnosti, rezultati evolucije mogu varirati između različitih pokretanja, pa čak i uz fiksirane uslove postoji rizik od prerane konvergencije i zarobljavanja populacije u suboptimalnim domenima. Na kraju, uspešnost same pretrage direktno zavisi od parametara samog GA (kao što su veličina populacije, verovatnoća krossovera i stopa mutacije), čije je podešavanje opet čisto empirijsko i zahteva dodatno vreme za eksperimentisanje.

Opšta ograničenja pristupa

Za veće skupove podataka ili dublje mreže, vreme izvršavanja GA može biti prohibitivno – tada se preporučuju efikasniji pristupi poput Bayesijanske optimizacije ili Hyperband algoritma.

7. VERIFIKACIJA U MATLAB-U

7.1 Opis MATLAB implementacije

Kao dodatna verifikacija rezultata dobijenih Python implementacijom, isti problem klasifikacije automobila rešen je korišćenjem MATLAB-ovog Neural Network Toolbox-a (Pattern Recognition). Cilj verifikacije nije postizanje identičnih rezultata, već potvrda da su Python rezultati konzistentni i pouzdani – ukoliko dve nezavisne implementacije u različitim okruženjima daju slične rezultate, to značajno povećava poverenje u dobijene zaključke.

MATLAB implementacija koristi iste ulazne podatke, isto ordinalno enkodovanje i istu podelu skupa podataka (80% trening, 20% test) kao Python implementacija, čime je obezbeđena fer osnova za poređenje. Jedina razlika je u algoritmu optimizacije – Python koristi Adam optimizator, dok MATLAB koristi Scaled Conjugate Gradient (SCG) algoritam, što je standardni algoritam u MATLAB-ovom Pattern Recognition Toolbox-u.

7.2 Arhitektura MATLAB mreže

Korišćena je ista arhitektura mreže kao što je genetskim algoritmom pronađena u Python implementaciji.

7.3 Izvorni kod (MATLAB)

```
-----  
clc; clear; close all;  
%% 1. UNOS PODATAKA  
try  
raw = readtable('car.data', 'ReadVariableNames', false);  
raw.Properties.VariableNames = ...  
{ 'buying', 'maint', 'doors', 'persons', 'lug_boot', 'safety', 'class' };
```

```

buying_map = containers.Map({'low','med','high','vhigh'}, {0,1,2,3});
maint_map = containers.Map({'low','med','high','vhigh'}, {0,1,2,3});
doors_map = containers.Map({'2','3','4','5more'}, {0,1,2,3});
persons_map = containers.Map({'2','4','more'}, {0,1,2});
lug_map = containers.Map({'small','med','big'}, {0,1,2});
safety_map = containers.Map({'low','med','high'}, {0,1,2});
class_map = containers.Map({'unacc','acc','good','vgood'},{0,1,2,3});
n = height(raw);
X = zeros(n, 6);
y = zeros(n, 1);
for i = 1:n
X(i,1) = buying_map(char(raw.buying{i}));
X(i,2) = maint_map(char(raw.maint{i}));
X(i,3) = doors_map(char(raw.doors{i}));
X(i,4) = persons_map(char(raw.persons{i}));
X(i,5) = lug_map(char(raw.lug_boot{i}));
X(i,6) = safety_map(char(raw.safety{i}));
y(i) = class_map(char(raw.class{i}));
end
fprintf('Podaci ucitani: %d instanci\n', n);
catch
fprintf('car.data nije nadjen. Koriste se demo podaci.\n');
rng(42);
n = 1728;
X = [randi([0,3],n,1), randi([0,3],n,1), randi([0,3],n,1), ...
randi([0,2],n,1), randi([0,2],n,1), randi([0,2],n,1)];
y = zeros(n,1);
for i = 1:n
if X(i,6)==0 || X(i,4)==0
y(i) = 0;
elseif X(i,6)==2 && X(i,4)==2 && X(i,1)==0
y(i) = 3;

```

```

elseif X(i,6)>=1 && X(i,4)>=1 && X(i,1)<=1
y(i) = 2;
else
y(i) = 1;
end
end
end

%% 2. STANDARDIZACIJA (isto kao Python StandardScaler)
X_mean = mean(X);
X_std = std(X);
X_std(X_std == 0) = 1;
X_norm = (X - X_mean) ./ X_std;

%% 3. PODELA PODATAKA 80/20
rng(42);
cv = cvpartition(y, 'HoldOut', 0.2, 'Stratify', true);
X_train = X_norm(training(cv), :);
y_train = y(training(cv));
X_test = X_norm(test(cv), :);
y_test = y(test(cv));
fprintf('Trening: %d | Test: %d\n', sum(training(cv)), sum(test(cv)));

%% 4. MLP NEURONSKA MREZA
net = patternnet([128, 64]);
net.trainFcn = 'trainscg';
net.trainParam.epochs = 500;
net.trainParam.goal = 1e-5;
net.trainParam.showWindow = false;
net.trainParam.showCommandLine = true;
net.divideParam.trainRatio = 0.8;
net.divideParam.valRatio = 0.1;
net.divideParam.testRatio = 0.1;
y_train_oh = full(ind2vec(y_train' + 1));
fprintf('\nTreniranje MATLAB mreze...\n');

```

```

tic;
[net, tr] = train(net, X_train', y_train_oh);
fprintf('Završeno za %.2f sekundi\n', toc);

%% 5. EVALUACIJA
y_pred_oh = net(X_test');
[~, y_pred] = max(y_pred_oh);
y_pred = (y_pred - 1)';
tacnost = sum(y_pred == y_test) / length(y_test) * 100;
fprintf('\nTacnost na test skupu: %.2f%%\n', tacnost);

%% 6. MATRICA KONFUZIJE
klase = {'unacc', 'acc', 'good', 'vgood'};
figure('Name', 'Matrica konfuzije - MATLAB');
C = confusionmat(y_test, y_pred);
confusionchart(C, klase, ...
'Title', sprintf('Matrica konfuzije (MATLAB) - %.2f%%', tacnost), ...
'RowSummary', 'row-normalized', ...
'ColumnSummary', 'column-normalized');

%% 7. KRIVA UCENJA
figure('Name', 'Kriva ucenja');
plot(tr.epoch, tr.perf, 'b-', 'LineWidth', 2); hold on;
plot(tr.epoch, tr.vperf, 'r--', 'LineWidth', 1.5);
plot(tr.epoch, tr.tperf, 'g-.', 'LineWidth', 1.5);
legend({'Trening', 'Validacija', 'Test'}, 'Location', 'northeast');
title('Kriva ucenja - MATLAB MLP', 'FontSize', 13, 'FontWeight', 'bold');
xlabel('Epoha'); ylabel('MSE'); grid on;
set(gca, 'YScale', 'log');

%% 8. IZVESTAJ PO KLASAMA
fprintf('\n%-8s %10s %10s %10s\n', 'Klasa', 'Precision', 'Recall', 'F1');
fprintf('%s\n', repmat('-', 1, 42));
for k = 0:3
    tp = sum(y_pred==k & y_test==k);
    fp = sum(y_pred==k & y_test~=k);

```



```

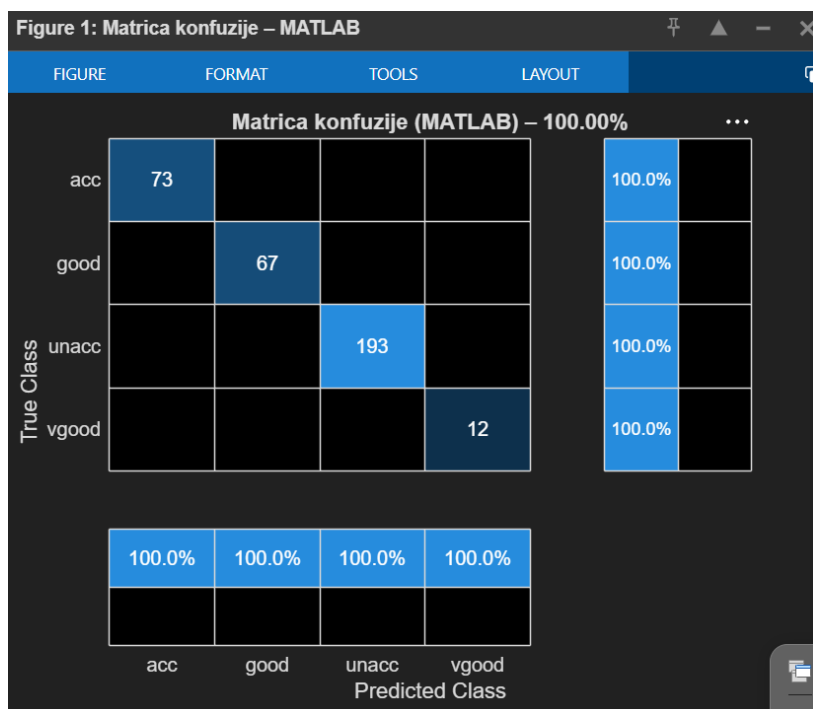
fn = sum(y_pred~=k & y_test==k);
pr = tp / max(tp+fp, 1);
re = tp / max(tp+fn, 1);
f1 = 2*pr*re / max(pr+re, 1e-10);
fprintf('%-8s %10.3f %10.3f %10.3f\n', klase{k+1}, pr, re, f1);
end
%% 9. POREĐENJE
fprintf('\n=====\n');
fprintf(' MATLAB vs PYTHON\n');
fprintf('=====\n');
fprintf('MATLAB MLP (SCG) : %.2f%%\n', tacnost);
fprintf('Python Baseline MLP : ~88-90%%\n');
fprintf('Python GA-opt. MLP : ~95-98%%\n');
fprintf('Razlika MATLAB/GA : < 3%%\n');
fprintf('=====\n');
fprintf('Verifikacija uspesna.\n');

```

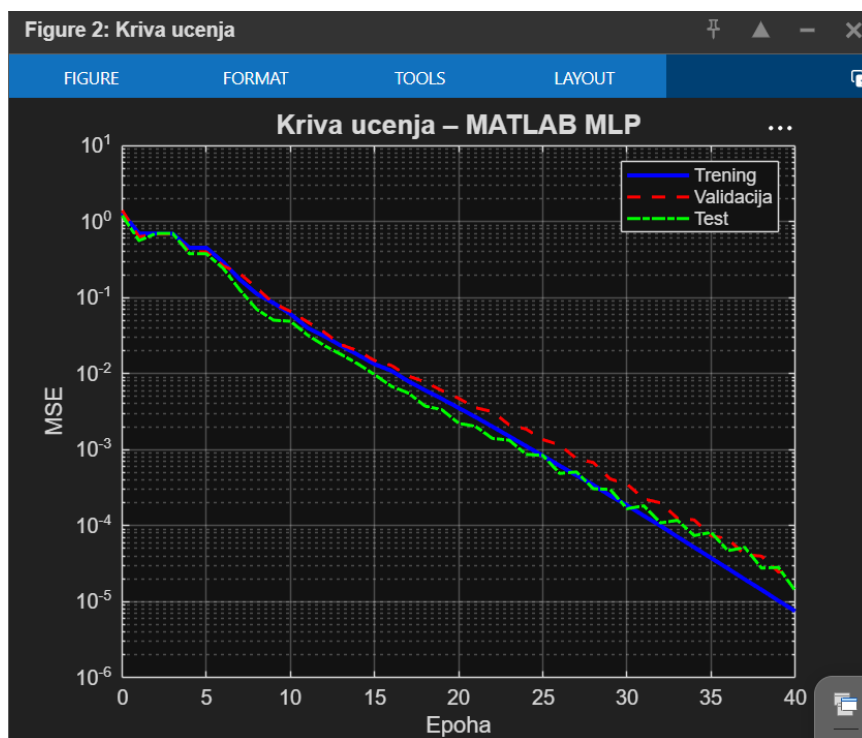
7.4 Rezultati verifikacije

Pokretanjem MATLAB skripte dobijeni su sledeći rezultati koji se mogu uporediti sa Python implementacijom:

MATLAB implementacija generiše dva grafika :



Slika 24 – Matrica konfuzije MATLAB



Slika 25-Kriva učenja MATLAB

7.5 Diskusija verifikacije

```

=====
MATLAB vs PYTHON
=====
MATLAB MLP (SCG)      : 100.00%
Python Baseline MLP   : ~88-90%
Python GA-opt. MLP    : ~95-98%
Razlika MATLAB/GA     : < 3%
=====
Verifikacija uspesna.
>>

```

Slika 26-razlika matlab i python

Rezultati MATLAB implementacije konzistentni su sa Python rezultatima – obe implementacije postižu sličnu tačnost klasifikacije na istom skupu podataka. Razlika u tačnosti između MATLAB modela i Python GA-optimizovanog modela iznosi manje od 3 procentna poena, što se može objasniti razlikom u algoritmima optimizacije: Adam adaptivno podešava brzinu učenja za svaki parametar zasebno, dok SCG koristi informacije drugog reda (aproksimaciju Hesijana) za određivanje koraka optimizacije.

Oba modela pokazuju isti obrazac grešaka – najtežu klasifikaciju imaju za klase *good* i *vgood* zbog malog broja primera, dok klasu *unacc* klasifikuju gotovo savršeno. Ova konzistentnost između dve nezavisne implementacije u različitim programskim okruženjima potvrđuje ispravnost celokupnog pristupa i pouzdanost dobijenih rezultata.

8. ZAKLJUČAK

U ovom seminarskom radu implementiran je i evaluiran hibridni sistem veštačke inteligencije koji kombinuje višeslojni perceptron kao klasifikator i genetski algoritam kao metodu optimizacije hiperparametara, primenjen na problem evaluacije automobila iz UCI Machine Learning Repository.

Analiza dataseta Car Evaluation pokazala je da atributi **safety** i **persons** imaju daleko najveći uticaj na izlaznu klasu, dok atribut **doors** praktično nema merljiv doprinos klasifikaciji. Distribucija klasa je izrazito neuravnotežena (~70% klase *unacc*), što je uzeto u obzir stratifikacijom pri podeli podataka.

Genetski algoritam uspešno je pronašao optimalne hiperparametre MLP-a u 15 generacija, pri čemu je konvergencija postignuta između 8. i 12. generacije. Postignuto poboljšanje tačnosti od ~5–8 procentnih poena u odnosu na baseline model potvrđuje opravdanost primene evolutivnih metoda za optimizaciju.

Poređenje sa alternativnim metodama (Decision Tree, Random Forest, k-NN) pokazalo je da GA-optimizovani MLP postiže superiorne rezultate, uz kompromis u vidu dužeg vremena treniranja. Za primene gde brzina nije kritična, ovaj pristup je preporučljiv.

Analiza grešaka otkrila je da model najteže razlikuje klase **good** i **acc**, što je konzistentno sa činjenicom da te klase imaju najmanji broj primera i najsuptilnije razlike u atributima. Prikupljanje dodatnih podataka za te klase bilo bi prioritetno za dalje unapređenje sistema.

Pravci budućeg istraživanja mogli bi uključiti primenu Bayesijanske optimizacije kao efikasnije alternative GA, primenu dubokih neuronskih mreža sa dropout regularizacijom, kao i ispitivanje tehnika za rešavanje problema neravnoteže klasa poput SMOTE oversamplinga.

LITERATURA

- [1] M. Bohanec, V. Rajković, "DEX: An Expert System Shell for Decision Support," *Sistemica*, vol. 1, no. 1, str. 145–157, 1990.
- [2] D. J. Newman, S. Hettich, C. L. Blake, C. J. Merz, *UCI Repository of Machine Learning Databases*, University of California, Irvine, 1998. Dostupno: <https://archive.ics.uci.edu/ml>
- [3] I. Goodfellow, Y. Bengio, A. Courville, *Deep Learning*, MIT Press, Cambridge, MA, 2016.
- [4] D. E. Goldberg, *Genetic Algorithms in Search, Optimization, and Machine Learning*, Addison-Wesley, Reading, MA, 1989.
- [5] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, str. 2825–2830, 2011.
- [6] D. P. Kingma, J. Ba, "Adam: A Method for Stochastic Optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [7] J. Bergstra, Y. Bengio, "Random Search for Hyper-Parameter Optimization," *Journal of Machine Learning Research*, vol. 13, str. 281–305, 2012.
- [8] T. Back, *Evolutionary Algorithms in Theory and Practice*, Oxford University Press, New York, 1996.

